

CPSC 250 Development Environment Setup

Christopher Newport University

Summer 2026

1. Purpose

In CPSC 250, students will regularly run, modify, and experiment with Python code examples provided throughout the course. To support this workflow, all students should configure a working Python development environment on their personal laptop.

This document walks you through:

- Installing Python
- Installing PyCharm
- Creating a GitHub account
- Forking the course repository
- Cloning the repository in PyCharm
- Creating a Python virtual environment
- Running example Python programs
- Using GitHub through PyCharm

2. Required Software

Students must install the following software:

- Python 3
- PyCharm Community Edition
- Git

3. Install Python

3.1 Windows

1. Go to:

<https://www.python.org/downloads/>

2. Download the latest version of Python 3.

3. IMPORTANT: During installation, check:

“Add Python to PATH”

4. Complete the installation.

3.2 macOS

1. Go to:

<https://www.python.org/downloads/>

2. Download the latest macOS installer.
3. Run the installer and complete installation.

4. Install PyCharm Community Edition

1. Go to:

<https://www.jetbrains.com/pycharm/download/>

2. Download:

PyCharm Community Edition

3. Install using the default settings.

5. Install Git

5.1 Windows

1. Go to:

<https://git-scm.com/download/win>

2. Download and install Git using the default settings.

5.2 macOS

Git is often already installed.

Open the Terminal application and type:

```
git --version
```

If Git is not installed, macOS will usually prompt you to install the command line developer tools automatically.

6. Create a GitHub Account

1. Go to:

<https://github.com>

2. Create a GitHub account if you do not already have one.
3. Verify that you can successfully log in.

7. Fork the Course Repository

The course repository contains lecture examples, sample programs, notes, and supporting materials used throughout the semester.

7.1 Course Repository

`https://github.com/brash99/cpsc250`

7.2 Fork Instructions

1. Log into GitHub.
2. Navigate to:
`https://github.com/brash99/cpsc250`
3. Click the “Fork” button near the upper-right corner.
4. GitHub will create your own personal copy of the repository.

8. Clone Your Fork Using PyCharm

1. Open PyCharm.
2. Select:
Get from VCS
3. Log into GitHub when prompted.
4. IMPORTANT: Select YOUR fork of the repository, NOT the instructor repository.
5. Choose a local directory for the project.
6. Click:

Clone

9. Create a Python Virtual Environment

PyCharm will usually offer to create a virtual environment automatically.

Accept the default settings.

If needed, you can manually create one later.

9.1 Windows PowerShell

```
python -m venv venv
```

Activate:

```
.\venv\Scripts\Activate.ps1
```

9.2 macOS Terminal

```
python3 -m venv venv
```

Activate:

```
source venv/bin/activate
```

10. Install Required Packages

If the repository contains a `requirements.txt` file, install packages using:

10.1 Windows PowerShell

```
pip install -r requirements.txt
```

10.2 macOS Terminal

```
pip3 install -r requirements.txt
```

11. Run an Example Program

1. Open one of the example Python files in the repository.
2. Right-click inside the editor window.
3. Select:

Run

4. Verify that the program executes successfully.

12. Make a Small Test Change

To verify that GitHub integration works correctly:

1. Open a Python file.
2. Add a simple comment such as:

```
# Testing my development environment
```

13. Commit and Push Using PyCharm

1. In PyCharm, select:

Git → Commit

2. Enter a commit message such as:

```
Test commit
```

3. Commit the changes.

4. Then select:

Git → Push

5. Verify that the changes appear in your GitHub repository.

14. Windows PowerShell vs Git Bash

These instructions use Windows PowerShell as the default Windows terminal environment because:

- PowerShell is installed by default on modern Windows systems.
- It integrates well with PyCharm.
- It works well with GitHub and GitHub CLI.
- It minimizes additional installation complexity.

Some other courses may use Git Bash instead. Either environment can work successfully for Python development.

15. Troubleshooting

15.1 Python Not Found

Verify installation:

```
python --version
```

or on macOS:

```
python3 --version
```

15.2 Git Not Found

Verify installation:

```
git --version
```

15.3 Push Authentication Problems

Use PyCharm's built-in GitHub login system rather than manually entering passwords or tokens.

16. Final Verification Checklist

Before class begins, verify that you can:

- Run Python programs
- Open the repository in PyCharm
- Commit changes
- Push changes to GitHub
- Pull updates from GitHub